# INDEX

NAME: SaiAkhilesh BTD: 2nd SEC: S9 ROLL NO: 1125Y4 1484 SUB: DSA observation

| S. No. | Date | Title | Page No. | Teacher's Sign / Remarks |
|---|---|---|---|---|
| 1. | 19/8/26 | Experiment: 1: implementation of Basic array operations (Insertion, Deletion, Traversal) | | |
| 2. | 19/8/26 | Experiment: 2: Building a Dynamic Employee Record System using structure pointers | | |
| 3. | 25/8/26 | 3: To implement of Basic stack operations (PEEK, PUSH, POP) | | |
| 4. | 9/9/26 | Experiment: 3: Implementation of stack and Expression conversion - Infix to postfix | | |
| 5. | 16/9/26 | Experiment: 4: Postfix Expression Evaluation using stack | | |

# INDEX

NAME: Salakhile Sri. U SEC: 3Y ROLL NO: 1[illegible] SUB: DSA Observation

| S. No. | Date | Title | Page No. | Teacher's Sign / Remarks |
|---|---|---|---|---|
| 1. | 23/9/26 | Experiment: 5. Implementation of simple Queue using Array | | |
| 2. | 30/9/26 | Experiment: 6: Implementation of priority Queue using unordered Array | | |
| 3. | 30/9/26 | Experiment: 7: Implementation of singly linked list (Insertion, Deletion and traversal) | | |
| 4. | 7/10/26 | Experiment: 8: Implementation of circular queue using linked list | | |
| 5. | 14/10/26 | Experiment: 9: Implementation and traversal (In-order, pre-order, post-order) of a Binary Search tree. | | |

NAME Srikhilesh Acharya

SEM 11th year DIV 99 ROLL NO 1129914464

SUBJECT Data structure & Algorithm observation

# INDEX

| SR NO | DATE | TITLE | PAGE NO | TEACHER'S SIGN |
|---|---|---|---|---|
| 10 | 21/10/24 | Ex:10: Implementation of Graph search Algorithms (BFS and DFS) | | |
| 11. | 21/10/24 | Ex:11: Implementation of Bubble, selection and Heap Sorts. | | |
| 12. | 28/10/24 | Ex:12: Implementation of Hashing with collision Resolution using linear probing | | |

classmate
Date 12/8/26
Page

# 1> Creation of an Array

* **Aim:-**

→ To create an array and display its elements

* **Algorithm:-**

1> Start
2> Read the number of elements
3> Read the array elements
4> Display the elements
5> stop

* **Program:-**

→
```
#include<stdio.h>
int main()
{
int a[10],n,i;
printf("Enter size");
Scanf("%d",&n);
Printf("Enter elements:");
for(i=0;i<n;i++)
  scanf("%d",&a[i]);
 Printf("array elements:");
  for(i=0;i<n;i++)
  printf("%d",a[i]);
  return 0;
}
```

* **Result:**

→ The array was created and displayed successfully

classmate
Date
Page

* Input :-

→ Enter size : 5
Enter elements : 10 20 30 40 5

* output :-

→ Array elements : 10 20 30 40 50

classmate
Date 19/8/25
Page

## 2) Insertion in an array

* Aim:

→ To insert an elements into an array.

* Algorithm:

i. Start
ii. Read array size and element
iii. Read position and new element
iv. Shift element to right
v. insert the new element
vi. Display the array
vii. Stop.

* Program:

→
```
#include<stdio.h>
int main() {
int a[10], n, i, pos, item;
scanf("%d", &n);
 for(i=0; i<n; i++) {
 scanf("%d", &a[i]);
}scanf("%d%d", &pos, &item);
 for(i=n; i>=pos; i--) {
   a[i] = a[i-1];
} a[pos-1] = item;
  for(i=0; i<=n; i++)
     printf("%d", a[i]);
     return 0;
     }
```

* Input:

→ 5
10 20 30 40 50
3 25

* Output:-

→ 10 20 ~~30~~ ~~25~~ 40 50

* Result:-

→ The element are inserted successfully

classmate
Date 19/8/25
Page

## 3> Deletion in an Array

* **Aim:**
→ To delete an element from an array

* **Algorithm:**
1> Start
2> Read array size and elements
3> Read the position
4> Shift elements to the left
5> Display the updated array
6> Stop

* **Program:**
→
```
#include <stdio.h>
int main() {
int a[10], n, i, pos;
scanf("%d", &n);
for (i=0; i<n; i++) {
 scanf("%d", &a[i]);
} scanf("%d", &pos);
 for (i = pos-1; i<n-1; i++){
    a[i] = a[i+1]
}   n--;
 for (i=0; i<n; i++) {
  printf("%d", a[i]);
} return 0;
}
```

* **Result:** The elements was inserted successfully

* Input :-

→ 5
10 20 30 40 50
3

* output :-

→ 10, 20, 40, 50

classmate
Date 19/8/26
Page

# 4> Traversal of an array

* **Aim :-**
To traverse and display all elements of an array

* **Algorithm :-**

1> Start
2> Read array size and elements
3> Display each elements
4> Stop.

* **Program :-**

→
```
#include <stdio.h>
int main() {
 int a[10], n, i;
 scanf ("%d", &n);
 for (i=0 ; i<n ; i++)
  scanf ("%d", &a[i]);
  for (i=0 ; i<n ; i++)
  printf ("%d", a[i]);
  return 0;
}
```

* **Result :-**

→ The array elements were traversal successfully.

* Input :-

→ 5
2 4 6 8 10

* Output :-

→ 2 4 6 8 10

classmate
Date 19/8/26
Page

# 5) linear search

* **Aim:**
→ To search an elements in an array using linear search

* **Algorithm:**

1) Start
2) Read array size and elements
3) Read the elements to search
4) compose the elements to search
5) if found, display its position otherwise display "not found"
6) Stop

* **Program:**
→

```
#include <stdio.h>
 int main() {
int a[10], n, i, key;
Scanf ("%d", &n);
for (i=0 ; i<n ; i++) {
Scanf ("%d", &a[i]);
}Scanf( "%d", & key);
    for (i=0 ; i<n ; i++) {
        if (a[i] == key) {
        Printf(" found at %d",i
          return 0;
        }
    }
```

```
Printf ("not found");
return 0;
}
```

* Result:
→ The element was searched successfully using linear search

* ~~Poi~~ Input:

→ 5
10 20 30 40 60
30
~~Out~~

* Output:

→ Found at 3

classmate
Date 19/8/26
Page

# Exercise -2- Building a Dynamic Employee Record System using Structure pointers

* Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_EMPLOYEES 100

struct Employee {
    int id;
    char name[50];
    float salary;
};

struct Employee *emp[MAX_EMPLOYEE
int count=0;

void addEmployee() {
if (count >= MAX_EMPLOYEES) {
printf("EMPLOyee list is full!\n"
return;
}
Struct Employee *e=(struct Emplo
        malloc(sizeof(struct Employe
Printf("Enter ID:");
scanf("%d", &e->id);
printf("Enter name:");
```

→ Output:

1. Add 2. Display 3. Search 4. Delete
5. Exit

choice : 2
No employees to show.

1. Add 2. Display 3. Search 4. Delete 5. E

choice : 1
Enter ID : 01
Enter Name : aaa
Enter salary : 10000
Employee added!

1. Add 2. Display 3. Search 4. Delete
5. Exit

Choice : 2

| ID | Name | Salary |
|---|---|---|
| 1 | aaa | 10000 |

1. Add 2. Display 3. Search 4. Delete 5. Exit

choice : 3
Enter ID to search : 02
Found : aaa, Salary : 10000

```
scanf("%s", e->name);
printf("Enter salary: ");
scanf("%f", &e->salary);
emp[count] = e;
count++;
printf("Employee added!\n\n");
}
void displayEmployees() {
if (count == 0) {
printf("No employees to show.\n\n");
return;
}
printf("\nID\tName\t\tSalary\n");
for(int i=0; i<count; i++) {
printf("%d\t%s\t\t%.2f\n",
emp[i]->id, emp[i]->name,
emp[i]->salary);
}
printf("\n");
}
void searchEmployee() {
int id;
printf("Enter ID to search: ");
scanf("%d", &id);
for(int i=0; i<count; i++) {
if (emp[i]->id == id) {
printf("Found: %s, salary: %.2f\n",
emp[i]->name, emp[i]->salary);
return;
```

```
}
if (emp[i]->id == id) {
free(emp[i]);
}
printf("Employee not found.\n\n");
}
Void deleteEmployee() {
int id;
printf("Enter ID to delete:");
scanf("%d", &id);
for (int i=0; i<count; i++)
if (emp[i]->id = id) {
free (emp[i]);
emp[i] = emp[count - 1];
count --;
printf("Employee deleted.\n\n");
return;
}
}
printf("Employee not found.\n")
}
void freeAll() {
for (int i=0; i<count; i++){
free (emp[i]);
}
count = 0;
}
```

```
int main() {
int choice;
do {
printf("1. Add 2. Display 3. Search
        4. Delete 5. Exit\n");
printf("choice:");
scanf("%d", &choice);
switch (choice) {
case 1: addEmployee(); break;
case 2: displayEmployee(); break;
case 3: searchEmployee(); break;
case 4: deleteEmployee(); break;
case 5: freeAll(); printf("Bye!\n");
        break;
Default: printf("Invalid choice.\n");
}
} while(choice != 5);
return 0;
}
```

classmate
Date 27/8/26
Page

# PUSH Operation

* Aim:
→ To insert an element into the stack (Push operation) while checking for a stack overflow condition.

* Algorithm:

- check if top == Max - 1
- If true, display "stack overflo and exit (the stack is full.
- otherwise, increment top by 1 (top = top + 1).
- Insert the new element into the stack at the top inde stack[top] = element.
- Display success message

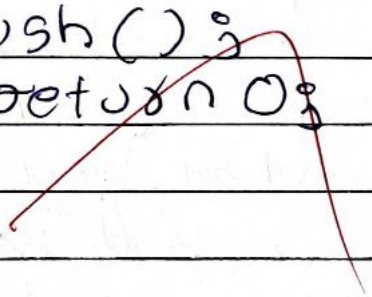
* Program

```
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = -1;
void push() {
    int val;
    if (top == Max - 1) {
        printf(" overflow: stack is f
```

* Output:-

Enter element to push : 10
pushed 10 into the stack.

```
} else {
    printf("Enter element to push:");
    scanf("%d", &val);
    top = top + 1;
    stack[top] = val;
    printf("pused %d into the
        the stack.\n", val);
    }
}
int main() {
    push();
    return 0;
}
```



classmate
Date 27/8/26
Page

# + POP operation

* **Aim**

To remove implement the pop operation on a stack using an array to remove the top element and handle stack underflow.

* **Algorithm**

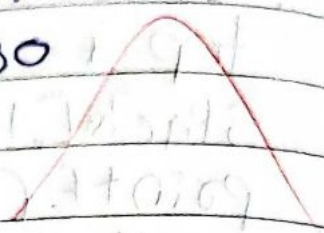
1- check if the stack is empty by evaluating if top == -1.
2- If True, display "stack underflow" and terminate the deletion.
3- If false:
- Retrieve the value stored at stack[top]
- Print or return the retrieved value
- Decrement top by 1 to effectively remove the element.

* **Program**

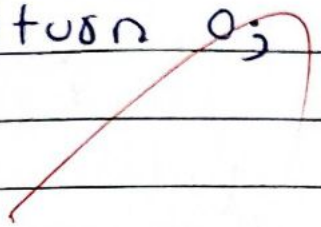
```
#include <stdio.h>
int stack[] = {10, 20, 30}
int top = 2;

void pop() {
    if (top == -1) {
        printf("Error: Stack underflow.\n
    } else {
        printf("Popped element: %d\n",
                                    stack[top]
```

* output

--- POP operation ----
Popped element: 30



```
    top--;
  }
}
int main() {
  printf("----POP OPERATION---");
    pop();
    return 0;
}
```



# PEEK Operation

* Aim:
→ To implement the peek operation on a stack using an array to view the current top element without changing the state of the stack

* Algorithm
1. Check if the stack is empty by evaluating if top == -1.
2. if True, display "Stack is Empty"
3. If False:
   - Print or return the value located at index stack[top].
   - Maintain the exact value of top without decrementing or incrementing

* Program:

```
#include <stdio.h>
int stack[] = {15, 25}
int top = 1;

void peek() {
    if (top == -1) {
        printf("Stack is empty.\n");
    }
```

* Output:

---- PEEK OPERATION ----

The top element (Peek) is : 25

```
else {
    printf("The top element (Peek) is: %d",
                                    Stack[top]);
    }
}
int main() {
    printf("---PEEK OPERATION---");
    Peek();
    return 0;
}
```

classmate
Date 9/9/26
Page

# Experiment No. 3 :-

## Implemenation of stack and Expression conversion - Infix to postfix

* **Aim :**

To write a C, program a stack using arrays (PUSH, POP, PEEK operation) and to convert a given infix expression into its equivalent postfix expression using a stack.

* **Algorithm**

1. initialize : create an empty stack and an empty postfix output string.
2. Scan : Read the infix expression from left to Right, character by character.
3. Process character:
   - operand (letter/Number) : Append it directly to the output.
   - left parenthesis ( ( ) : Push it onto the stack
   - Right Parenthesis ( ) ) : Pop and append operators to the output until a ( is popped. Discard the (.
   - operator ( +, -, *, /, etc. ) : Pop and append operators from the stack

that have higher or equal precedence, then push the current operator.

4. Finalise: Once scanning is complete, pop all remaing operators from the stack and append them to the output.

5. Output: Display the final postfix expression.

* Program:

```
#include <stdio.h>
# include <string.h>
# include <ctype.h>

# define MAX 100
 char stack[MAX];
int top = -1;
void push (char c) {
      if (top == MAX - 1)
        printf ("stack overflow\n");
      else
          stack[++top] = c;
  }
  char pop() {
      if (top == -1) {
        printf("stack underflow\n");
```

```
    return -1;
}
char peek() {
    if (top == -1)
        return -1;
    return stack[top];
}
int precedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return -1;
}
int isOperand(char c) {
    return isalnum(c);
}
void infixToPostfix(char infix[],
        char postfix[]) {
    int i, j=0;
    for (i=0; infix[i] != '\0'; i++) {
        char c = infix[i];
        if (isOperand(c)) {
            postfix[j++] = c;
        } else if (c == '(') {
            push(c);
        } else if (c == ')') {
            while (top != -1 && peek()
                != '(')
```

```
        postfix[j++] = pop();
        pop();
    } else {
    while (top != -1 && precedence
        (peek()) >= precedence(c))
            postfix[j++] = pop();
        push(c);
    }
}
while (top != -1)
    postfix[j++] = pop();
    postfix[j] = '\0';
}
int main() {
    char infix[MAX], postfix[MAX];
    printf("Enter an infix expression:");
    scanf("%s", infix);
    infixTo postfix(infix, postfix);
    printf("postfix expression: %s\n",
            postfix);
    return 0;
}
```

* **Result:**

→ Thus, a C program was written and executed successfully to implement a stack and convert an infix expression to its postfix form.

* Output :

Enter an infix expression : A+B*(C-D)/E

postfix expression : ABCD-*E/+

classmate
Date 16/9/25
Page

# Experiment NO. 4 :-
## Postfix Expression Evaluation using Stack

* **Aim:**
To write a C program to evaluate a given postfix expression using a stack.

* **Algorithm:**
1. Start the program and declare a stack of type float with top initialized to -1.
2. Read the postfix expression as a string
3. Scan the postfix expression from left to right, one character at a time
4. If the scanned character is a digit (operand), convert it to a number and PUSH it onto the stack
5. If the scanned character is an operator. POP the top two elements from the stack; let the first popped value be op2 and the second be op1.
6. Apply the operator on op1 and op2 (op1 operator op2) and PUSH the result back onto the stack
7. Repeat steps 3 to 6 until all the characters of the postfix

expression are scanned.

8. After the scan is complete, Pop the final result from the stack; this is the value of the expression

9 Display the result and stop the program

* Program *

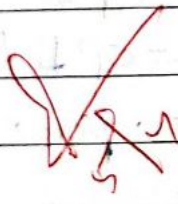
```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <math.h>

#define MAX 100

float stack[MAX];
int top = -1;
void push(float val) {
    if(top == MAX-1)
      printf("stack Overflow\n");
    else
      stack[++top] = val;
    }
    float pop() {
      if (top == -1) {
        printf("Stack Underflow");
        return 0;
      }
```

```
    return stack[top--];
}
float evaluatePostfix(char postfix[]){
    int i;
    float op1, op2, result;
for (i=0; postfix[i] != '\0'; i++){
    char c = postfix[i];

    if (isdigit(c)){
        push(c - '0');
    } else if(isspace(c)){
        continue;
    } else {
        op2 = pop();
        op1 = pop();
        switch(c){
        case '+': result = op1 + op2; break;
        case '-': result = op1 - op2; break;
        case '*': result = op1 * op2; break;
        case '/': result = op1 / op2; break;
        case '^': result = pow(op1, op2);
                  break;
        default:
            printf("Invalid operator: %c\n", c
                  );
            result = 0;
    }
        push(result);
```

```
    }
  }
    return pop();
}
int main () {
    char postfix[MAX];
printf("Enter a postfix expression (Sing
        digit operands) : ");
scanf("%s", postfix);
printf("Result = %.2f\n", evaluatePos
                - fix(postfix));
    return 0;
}
```

* Result:

Thus, a 'C' program was written and executed successfully to evalua a postfix expression using a sta



classmate
Date 23/9/25
Page

## EX. NO. 5: Implementation of Simple Queue using Array

* Aim :

→ To write a C program to implement a simple (linear) queue using an array and to perform ENQUEUE, DEQUEUE, and display operation.

* Algorithm:

1. Step 1: Start the program and declare an array[MAX] with front and rear initialized to -1.
2. Step 2: ENQUEUE(x): if rear equals MAX - 1 report QUEUE Overflow. otherwise, if front equals -1 set front to 0, increment rear and store x at queue[rear].
3. Step 3: DEQUEUE(): if front equals -1 or front is greater than rear, report Queue underflow.
otherwise, display the element at queue (front). increment

front, and if front exceeds rear reset both front and rear to -1.

4. Step 4: Display: if the queue is not empty, print all elements from index front to rear.
5. Step 5: Display a menu with options: Enqueue, Dequeue, Display and Exit.
6. Step 6: Repeat step 5 based on user choice until Exit is selected.
7. Step 7: Stop the program

classmate
Date
Page

* Program

```
#include <stdio.h>
#define MAX 5
int queue[MAX];
int front = -1, rear = -1;
void enqueue(int val) {
  if (rear == MAX - 1) {
    printf("Queue Overflow\n");
    return;
  }
  if (front == -1)
    front = 0;
  queue[++rear] = val;
  printf("%d enqueued to
      queue.\n", val);
}
void dequeue() {
  if (front == -1 || front > rear
                        {
    printf("Queue underflow\n")
    return;
  }
  printf("%d dequeued from
      queue.\n", queue[front]);
  front++;
  if (front > rear)
    front = rear = -1;
}
```

Output:

Simple Queue Menu
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
10 enqueued to queue.

Simple Queue Menu
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 20
20 enqueued to queue.

Simple Queue Menu
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 30
30 enqueued to queue.

```
void display() {
    int i;
    if (front == -1 || front > rear)
    {
        printf("Queue is empty.\n");
        return;
    }
    printf("Queue elements : ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}
int main() {
    int choice, val;
    do {
        printf("\n Simple Queue
                Menu ---\n");
        printf("1. Enqueue\n 2. Dequeue
            \n 3. Display \n 4. Exit\n");
        printf("Enter your choice:");
        scanf("%d", &choice);

        switch(choice) {
        case 1:
            printf("Enter value to enqueue:");
            scanf("%d", &val);
            enqueue(val);
            break;
```

- - Simple Queue Menu - -
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice : 3
Queue elements : 10 20 30

- - Simple Queue Menu - -
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice : 2
10 dequeued from queue

- - Simple Queue Menu - -
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice : 3
Queue elements : 20 30

```
case 2:
    dequeue();
    break;
case 3:
    display();
    break;
case 4:
    printf("Exiting program.\n");
    break;
default:
    printf("Invalid choice.\n");
}
} while (choice != 4);
return 0;
}
```

... Simple queue Menu ...
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting program

classmate
Date 3/09/25
Page

## EX. NO. 6: Implementation of Priority Queue using Unordered Array

* **Aim:**

→ To write a C program to implement a priority Queue using an unordered array and to perform findMin, Insert and Delete Min operations, where the element with the minimum value has the highest priority.

* **Algorithm:**

1. Step 1: Start the program and declare an array pq[MAX] with count n initialized to 0.

2. Step 2: Insert(x): if the array is full report overflow, otherwise store x at pq[n] and increment n.

3. Step 3: FindMin(): Scan the entire array pq[0.... n-1] and find the index of the smallest element; return that index

4. Step 4: DeleteMin(): call FindMin() to get the index of the minimum element, display it, then shift all subsequent elements one place to the left and decrement n.

5. Step 5: Display : print all the elements currently presed in the array.

6. Step 6: Display a menu with options Insert, Find Min, Delete Min, Display and Exit.

7. Step 7: Repeat step 6 based on user choice until Exit is selected

8. Step 8: Stop the program.

* Program :

```
#include<stdio.h>
#define MAX 100

int pq[MAX];
int n = 0;

void insert(int val) {
    if (n == MAX) {
        printf("Priority Queue is
                full.\n");
        return;
    }
    pq[n++] = val;
    printf("%d inserted into priority
            queue.\n", val);
}
int findMinIndex() {
    int i, minIndex = 0;

for (i = 1; i < n; i++)
    if (pq[i] < pq[minIndex])
        minIndex = i;
    return minIndex;
}
```

* Output:

Priority Queue Menu

1. Insert
2. Find Min
3. DeleteMin
4. Display
5. Exit

Enter your choice : 1
Enter value to insert : 40
40 inserted into priority q

Priority Queue Menu

1. Insert
2. Find Min
3. Delete Min
4. Display
5. Exit

Enter your choice: 1
Enter value to insert: 10
10 inserted into priority queu

priority Queue Menu

1. Insert
2. Find Min
3. Delete Min
4. Display
5. Exit

Enter your choice: 1

```
void findMin() {
    if (n == 0) {
    Printf("Priority Queue is
            Empty.\n");
    return;
  }
printf ("Minimum element (highest
        priority): %d\n", pq[findM-
        -in Index()]);
}
void deleteMin() {
   int i, minIndex;
   if (n == 0) {
    printf("Priority Queue is empty.
          Cannot delete.\n");
          return;
   }
 minIndex = findMin Index();
 printf("Deleted min element: %d\n
        pq[minIndex]);
 for (i=minIndex; i<n+1; i++)
       pq[i] = pq[i+1];
       n--;
 }
void display() {
    int i;
   if(n == 0) {
printf("Priority Queue is emty.\n
```

Enter value into insert: 25
25 inserted into priority queue

Priority Queue Menu
1. Insert
2. Find Min
3. Delete Min
4. Display
5. Exit
Enter your choice: 4
priority Queue elements: 40 10 25

Priority Queue Menu
1. Insert
2. Find Min
3. Delete Min
4. Display
5. Exit
Enter your choice: 2
Minimum element (highest priority): 10

Priority Queue Menu
1. Insert
2. Find Min
3. Delete Min
4. Display
5. Exit
Enter your choice: 3
Minimum element (highest priority): 10

```
    return;
}
printf("Priority Queue elements: ");
for (i = 0; i < n; i++)
    printf("%d", pq[i]);
printf("\n");
}
int main() {
    int choice, val;
    do {
        printf("\n Priority Queue Menu");
        printf("1. insert \n 2. FindMin \n
                3. DeleteMin \n 4. Display \n
                5. Exit \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert:");
                scanf("%d", &val);
                insert(val);
                break;
            case 2:
                findMin();
                break;
            case 3:
                deleteMin();
                break;
```

- Priority Queue Menu

1. Insert
2. Find Min
3. Delete Min
4. Display
5. Exit

Enter your choice: 3
Deleted min. element: 10

Priority Queue Menu

1. Insert
2. Find Min
3. Delete Min
4. Display
5. Exit

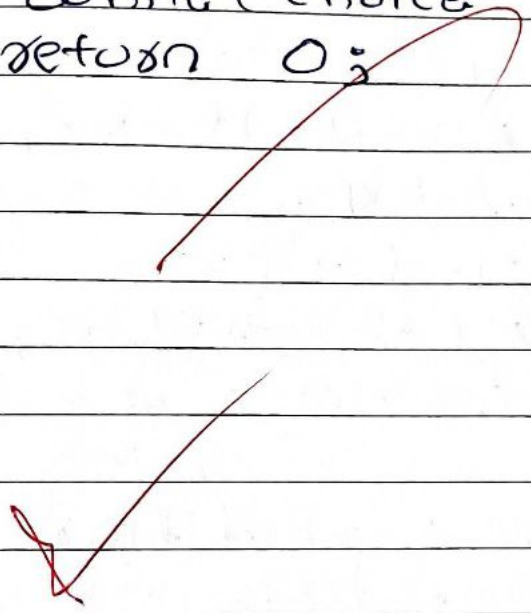
Enter your choice: 4
priority Queue elements: 40 25

Priority Queue Menu

1. Insert
2. Find Min
3. Delete Min
4. Display
5. Exit

Enter your choice: 5
Exiting program

```
case 4:
    display();
    break;
case 5:
    printf("Exiting program.\n");
    break;
default:
    printf("Invalid choice.\n");
}
} while(choice != 5);
return 0;
}
```



classmate
Date 30/9/26
Page

## EX. NO. 7: Implementation of singly linked list (Insertion, Deletion and Traversal)

* **Aim**

→ To write a C program to implement a singly linked list and perform insertion (at the beginning, at a given position, and at the end), deletion (at the beginning, at a given position, and at the end) and traversal of the list.

* ~~Impl~~ **Algorithm:**

1. Step 1: start the program and define a structure Node initialize head = NULL.
2. Define a structure Node with data and next
3. Insert at Beginning: create a node and link it before head
4. Insert at End: Create a node and link it to the last node
5. Insert at position: insert a node at the specified position.
6. Delete at ~~position~~ Beginning: Remove and free the last node.

7. Delete at End : Remove and free the last node.
8. Delete at position: Remove and free the node at the specified position.
9. Traverse : Display all node values from Head to NULL.
10. Display a menu and repeat operations until the user selects Exit.
11. Stop the Program

* Program

```
#include <stdio.h>
#include <stdlib.h>
Struct Node {
    int data;
    Struct Node * next;
};
Struct Node *head = NULL;
void insertBeginning (int val) {
    Struct Node * newNode = (struct
    Node *) malloc( sizeof(struct Node
    newNode -> data = val;
    newNode -> next = head;
    head = newNode;
}
```

* Output:

Singly linked list Menu
1. Insert at Beginning
2. Insert at position
3. Insert at End
4. Delete at Beginning
5. Delete at position
6. Delete at End
7. Traverse
8. Exit

Enter your choice: 3
Enter value: 10

Enter your choice: 3
Enter value: 20

Enter your choice: 3
Enter value: 30

Enter your choice: 2
Enter value and position (0-indexed)

Enter your choice: 7
linked list: 10 -> 15 -> 20 -> 30 -> N

Enter your choice: 5
Enter position (0-indexed): 1

```
void insertEnd(int val) {
  struct Node * newNode = (struct Node*)
      malloc(sizeof(struct Node));
  newNode -> data = val;
  newNode -> next = NULL;
  if (head == NULL) {
      head = newNode;
      return;
  }
  struct Node * temp = head;
  while (temp -> next != NULL)
      temp = temp -> next;
  temp -> next = newNode;
}

void insertposition(int val, int pos) {
  if (pos == 0) {
      insertBeginning(val);
      return;
  }
  struct Node * newNode = (struct Node*)
      malloc(sizeof(struct Node));
  new Node -> data = val;
  struct Node * temp = head;
  int i;
  for(i=0; i < pos - 1 && temp != NULL; i++)
      temp = temp -> next;
```

Enter your choice: 7
linked list: 10 -> 20 -> 30 -> NULL

Enter your choice: 8
Exiting program

```
if (temp == NULL) {
  printf("Invalid position.\n");
   free(newNode);
   return;
}
newNode -> next = temp -> next;
temp -> next = newNode;
}
void deleteBeginning() {
   if (head == NULL) {
   printf("List is empty.\n");
    return;
}
struct Node * temp = head;
 head = head -> next;
 free(temp);
}
void deleteEnd() {
      if (head == NULL) {
   printf("List is empty.\n");
    return;
}
if (head -> next == NULL) {
     free(head);
    head = NULL;
    return;
}
struct Node * temp = head;
```

```
while(temp -> next -> next != NULL)
    temp = temp -> next;
  free(temp -> next);
  temp -> next = NULL;
}
void deletePosition(int pos) {
  if (pos == 0) {
      deleteBeginning();
      return;
  }
struct Node *temp = head;
int i;
for(i=0; i < pos-1 && temp != NULL
    ; i++)
  temp = temp -> next;
if (temp == NULL || temp -> next ==
    NULL) {
  printf("Invalid position.\n");
  return;
}
struct Node *toDelete = temp -> next;
temp -> next = toDelete -> next;
free(toDelete);
}
void traverse() {
  struct Node *temp = head;
  if(head == NULL) {
  printf("List is empty.\n");
```

```
    return 1;
}
printf("Linked list: ");
while (temp != NULL) {
    printf("%d -> ", temp -> data);
    temp = temp -> next;
}
printf("NULL\n");
}
int main() {
    int choice, val, pos;
    do {
        printf("--- Singly linked list Menu ---\n");
        printf("1. Insert at Beginning\n 2. Insert at Position\n 3. Insert at End\n");
        printf("4. Delete at Beginning\n 5. Delete at position\n 6. Delete at End\n");
        printf("7. Traverse\n. 8. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        return 0;
    }
```

* **Result:**

⇒ Thus, a C program was written and executed successfully to implement insertion, deletion, and traversal operation on a singly linked list

classmate
Date 7/10/25
Page

# Experiment NO. 8:-
## Implementation of circular Queue using linked list

* **Aim :-**
→ To write a C program to implement a circular queue using a singly linked list and to perform ENQUEUE, DEQUEUE, and display operations.

* **Algorithm :-**
→ Step 1 :- Start & Define Node
- Define structure Node with an integer data and a pointer next.
- initialize a pointer rear to NULL.

Note: rear -> next always tracks the front node.

→ Step 2 :- Enqueue(x)
- Allocate a new node with data x.
- if empty (rear == NULL) : set new_node -> next = new_node and rear = new_node.
- otherwise : set new_node -> next = rear -> next, then rear -> next = new_node, and update rear = new_node

→ Step 3 :- Dequeue()
- if empty (rear == NULL) : Print "Queue underflow"

- if one a otherwise: Set front = rear -> next and print its data.
- if one node (front == rear):
  set rear = NULL.
- otherwise: Link rear -> next = front -> next.
- free the front node.

→ Step 4: Display()
- if empty(rear = NULL): Print a queue empty message.
- otherwise: Set temp = rear -> next(front)
  loop, printing temp -> data and moving temp = temp -> next, until temp equals the front node again.

→ Step 5: Menu & Repeat
- loop a menu offering Enqueue, Dequeue, Display, and Exit until Exit is chosen,

→ step 6:
- End the program.

classmate
Date
Page

program

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
        int data;
        struct Node *next;
    };
struct Node *rear = NULL;

void enqueue(int val) {
 struct Node *newNode = (struct Node*)
          malloc(sizeof(struct Node));
 newNode -> data = val;
 if (rear == NULL) {
    newNode -> next = newNode;
    rear = newNode;
  } else {
    newNode -> next = rear -> next;
    rear->next = newNode;
    rear = newNode;
  }
  printf("%d enqueued to circular
          queue.\n", val);
}
Void dequeue() {
    if (rear == NULL) {
      printf("Queue underflow\n");
      return;
}
```

* Output

Circular Queue

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1
Enter value to enqueue: 10
10 enqueue to circular queue.

Enter your choice: 1
Enter value to enqueue: 20
20 enqueue to circular queue

Enter your choice: 1
Enter value to enqueue: 30
30 enqueue to circular queue.

Enter your choice: 3
circular Queue elements: 10 20 30

Enter your choice: 2
10 dequeued from circular queue

Enter your choice: 3
circular Queue elements: 20 30
Enter your choice: 4
Exiting program.

```
stouct Node *front = rear->next;
printf("%d dequeued from circular
    queue.\n", front->data);
if (front == rear) {
    rear = NULL;
} else {
    rear->next = front->next;
}
free (front);
}
void display(): {
    if (rear == NULL) {
    printf("Queue is empty.");
    return;
}
struct Node *temp = rear->next;
printf("circular Queue elements:");
do {
    printf("%d", temp->data);
    temp = temp->next;
} while (temp != rear->next);
print("\n");
}
int main() {
    int choice, val;
do {
printf("----Circular Queue(linked list)");
printf("1. Enqueue\n 2. Dequeue\n
    3. Display\n 4. Exit\n");
```

```
print ("Enter your choice:");
scanf("%d", &choice);
switch(choice) {
  case 1:
  printf("Enter value to enqueue:");
  scanf("%d", &val);
  enqueue(val);
  break;
  case 2:
    dequeue();
    break;
  case 3;
    display();
    break;
  case 4;
    printf("Exiting program.\n");
    break;
  default:
    printf("Invalid choice.\n");
  }
} while (choice != 4);
return 0;
}
```

## Experiment NO. 9 :-

### Implementation and Traversal (In-order, Pre order, Post-order) of a Binary Search Tree

**Aim:**

To write a C program to implement a Binary Search Tree (BST) and to perform insertion and traversal of the tree using in-order, pre-order, post-order and level-order techniques.

**Algorithm:**

Step 1: Start the program and define a structure Node containing data, and pointers left and right, both initialized to NULL for a new node.

Step 2: Insert(root, value): if root is NULL, create and return a new node with the given value. if value is less than root's data, recursively insert into the left subtree; if greater, recursively insert into the right subtree.

Step 3: In-order Traversal: recursively traverse the left subtree, visit the root node, then recursively traverse the right subtree

Step 4: Pre-order Traversal: visit the root node, recursively traverse the left subtree, then recursively traverse the right subtree

Step 5: Post-order Traversal: recursively traverse the left subtree, then visit the right subtree, then visit the root node.

Step 6: level-order Traversal: insert the root into a queue; while the queue is not empty, remove a node, display it, and insert its non-null left and right children into the queue.

Step 7: Read the number of nodes and their values, and build the BST by repeatedly calling Insert.

Step 8: Display a menu with the traversal options and an option to insert new nodes; repeat until Exit is selected.

Step 9: Stop the program

program :

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node * left, * right;
};

struct Node *createNode(int val) {
    struct Node *newNode =
    (struct Node *) malloc(sizeof(struct Node));
    newNode -> data = val;
    newNode -> left = newNode -> right = NULL;

    return newNode;
}
struct Node *insert(struct Node *root,
                    int val) {
    if (root == NULL)
        return createNode(val);
    if (val < root -> data)
        root -> left = insert(root -> left, val);
else if (val > root -> data)
    root -> right = insert(root -> right, val);
    return root;
}
void inorder(struct Node *root) {
    if (root != NULL) {
```

* Output:

```
Enter number of nodes to insert
Enter 6 values: 50 30 70 20
                40 60

---- BST Traversal Menu
1. In-order
2. Pre-order
3. Post-order
4. level-order
5. Insert new node
6. Exit

Enter your choice: 1
In-order Traversal: 20 30 40
                    60 70

Enter your choice: 2
pre-order Traversal: 50 30 20
                     40 70 60

Enter your choice: 3
post-order Traversal: 20 40 3
                      60 70 50

Enter your choice: 4
level-order Traversal: 50 30 7
                       20 40 60
```

```
    inorder(root->left);
    printf("%d", root->data);
    inorder(root->right);
    }
}

void preorder(struct Node* root){
    if (root != NULL) {
    printf("%d", root->data);
    preorder(root->left);
    preorder(root->right);
    }
}
void postorder(struct Node * root){
    if (root != NULL) {
    postorder(root->left)
    postorder(root->right);
    printf("%d", root->data);
    }
}
#define QMAX 100
void levelorder(struct Node * root){
    struct Node * queue[QMAX];
    int front = 0, rear = 0;

    if (root == NULL) {
        printf("Tree is empty.\n");
        return;
    }
```

Enter your choice :6
Exiting program.

```
queue[rear++] = root;
while (front < rear) {
    struct Node * curr = queue[front++];
    return 3
2   printf("%d", curr->data);
    if (curr -> left != NULL)
        queue [rear++] = curr ->left;
    if (curr -> right != NULL)
        queue[rear++] = curr -> right;
    3
}
int main() {
    struct Node * root = NULL;
    int n, i, val, choice;

    printf("Enter number of nodes to
        insert : ");
    scanf("%d", &n);
    printf("Enter %d values :", n);
    for (i=0; i<n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }
do {
    printf(" BST Traversal Menu ");
    printf("1. In-order\n 2. Pre-order
    \n 3. Post-order \n 4. level-
    order\n 5. Insert new node\n 6. Exit
    \n");
```

```
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
    case 1:
    printf("In-order Traversal: ");
    inorder(root);
    printf("\n");
    break;
case 2:
    printf("Pre-order Traversal: ");
    preorder(root);
    printf("\n");
    break;
case 3:
    printf("Post-order Traversal: ");
    postorder(root);
    printf("\n");
    break;
case 4:
    printf("level-order Traversal: ");
    levelorder(root);
    printf("\n");
    break;
case 5:
    printf("Enter value to Insert: ");
    scanf("%d", &val);
    root = insert(root, val);
    break;
```

```
case 6:
    printf("Exiting program.\n");
    break;
default:
    printf("Invalid choice.\n");
} while (choice != 6);
return 0;
```

Page No.
Date 21/10/26

# Experiment NO. 10 - Implementation of Graph Search Algorithms (BFS & DFS)

* **Aim:**

→ To write a C program to represent a graph using an adjacency matrix and to traverse it using the Breadths-First Search (BFS) and Depth

* **Algorithm:**

1. Start the program and read the number of vertices n and edges. e
2. Read each edge (u,v) and set adj[u][v] = adj[v][u] = 1 in the adjacency matrix.
3. BFS (start): Restarted visited [] to 0. Mark start as visited and insert it into a queue. while the queue is not empty, dequeue a vertex, display it, and enqueue all its unvisited adjacent vertices (marking them visited).
4. DFS (v): Mark vertex v as visited and display it. Recursively call DFS(i) for every unvisited vertex i.
5. Read the starting vertex and display a menu with options 1. BFS, 2. DFS, 3. Exit; repeat based on user choice
6. Stop the program.

* Program :-

```
#include <stdio.h>
int adj[20][20], visited[20], n;

void bfs(int start) {
    int q[20], f=0, r=0;
    for(int i=0; i<n; i++) visited[i]=0;
    visited[start] = 1;
    q[r++] = start;
    printf("BFS Traversal: ");
    while (f<r) {
        int curr = q[f++];
        printf("%d", curr);
        for (int i=0; i<n; i++)
        if (adj[curr][i] && !visited[i])
            visited[i] = 1;
            q[r++] = i;
        }
    }
    printf("\n");
}
void dfs(int v) {
    visited[v] = 1;
    printf("%d", v);
    for (int i=0; i<n; i++)
        if (adj[v][i] && ! visited[i] dfs(i
}
int main() {
    int edges, u, v, start, ch;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter number of edges: ");
```

* Sample output:

```
Enter number of vertices: 5
Enter number of edges: 5
Enter edges (u v) for an undirected graph:
0 1
0 2
1 3
2 4
3 4
Enter Stating vertix: 0

--- Graph Traversal Menu ---
1. BFS
2. DFS
3. Exit
Enter your choice: 1
BFS Traversal: 0 1 2 3 4

--- Graph Traversal Menu ---
1. BFS
2. DFS
3. Exit
Enter your choice: 2
DFS Traversal: 0 1 3 4 2

--- Graph Traversal Menu ---
1. BFS
2. DFS
3. Exit
Enter your choice: 3
Exiting program
```

```
scanf("%d", &edges);
printf("Enter edges (u v) for an undirected
        graph:\n");
for (int i=0; i<edges; i++) {
    scanf("%d %d", &u, &v);
    adj[u][v] = adj[v][u] = 1;
}
printf("Enter starting vertex: ");
scanf("%d", &start);
do {
    printf("\n - Graph Traversal Menu ---
    \n 1. BFS \n 2. DFS \n 3. Exit \n Enter
    your choice: ");
    scanf("%d", &ch);
    if(ch == 1) bfs(start);
    else if (ch == 2) {
        for (int i=0; i<n; i++) visited
            visited[i] = 0;
        printf("DFS Traversal: ");
        dfs(start);
        printf("\n");
    } else if (ch == 3) printf("Exiting
        program.\n");
    else printf("Invalid choice.\n");
} while (ch != 3);
return 0;
}
```

* **Result:**

⇒ Thus, a C program was written and executed successfully to traverse a graph using BFS and DFS.

Page No.
Date 21/10/26

## Exp: 11: Implementation of Bubble, selection and Heap Sorts

* **Aim:**
To write a Cprogram to sort a given list of elements using Bubble sort, Selection Sort, and Heap Sort techniques.

* **Algorithm:**
1. start the program and read n elements into an array.
2. Bubble Sort: Repeatedly compare adjacent elements a[j] and a[j+1] and swap them if a[j] > a[j+1], for n-1 passes
3. Selection sort: for each position i from 0 to n-2, find the index min of the smallest element from i to n-1 and swap a[i] with a[min].
4. Heap sort:
   - Build-Max-Heap: call heapify() from index n/2 -1 down to 0
   - Extract Elements: Repeatedly swap root a[0] with last element a[i] and call heapify(a, i, 0) on the reduced heap.
5. Display a menu with options and print the sorted array after each choice
6. Stop the program.

* Program

```
#include <stdio.h>
void swap(int *a, int *b) {int t = *a;
  *a = *b; *b = t; }

void printArray(int a[], int n ){
  for (int i=0; i<n; i++) {
  printf("%d", a[i]);
  printf("\n");
 }
void bubble sort(int a[], int n ] {
  for (int i=0; i<n-1; i++)
   for (int j=0; j<n-i-1; j++)
    if (a[j] > a[j+1])
      swap(&a[j], &a[j+1]);
    }
void selection sort(int a[], int n ) {
  for (int i=0; i<n-1; i++) {
   int min=i;
   for(int j=i+1; j<n; j++)
     if (a[j] < a[min]) min = j;
   swap(& a[min], & a[i]);
 }
}
void heapify (int a[], int n, int i ){
  int max=i, l = 2 * i+1, r= 2*i+2;
 if (l < n && a[l] >a[max]) max=l;
 if (r<n && a[r] > a[max]) max=r;
  if(max != i) {
     swap(&a[i], &a[max]);
      heapify (a, n, max);
  }
}
```

* Program

```
#include <stdio.h>
void swap(int *a, int *b) {int t = *a;
  *a = *b; *b = t;}

void printArray(int a[], int n) {
  for (int i=0; i<n; i++) {
  printf("%d", a[i]);
  printf("\n");
}
void bubble sort(int a[], int n ) {
  for (int i=0; i<n-1; i++)
    for (int j=0; j<n-i-1; j++)
      if (a[j] > a[j+1])
        swap(&a[j], &a[j+1]);
    }
void selection sort(int a[], int n) {
  for (int i=0; i<n-1; i++) {
    int min=i;
    for (int j=i+1; j<n; j++)
      if (a[j] < a[min]) min = j;
    swap(&a[min], &a[i]);
  }
}
void heapify (int a[], int n, int i ) {
  int max = i, l = 2*i+1, r = 2*i+2;
  if (l<n && a[l] > a[max]) max=l;
  if (r<n && a[r] > a[max]) max=r
  if (max != i) {
    swap(&a[i], &a[max]);
    heapify (a, n, max);
  }
}
```

* Output

Enter number of elements :6
Enter 6 elements: 45 12 78 3 89 21

-- Sorting Menu --
1. Bubble sort
2. Selection sort
3. Heap sort
4. Exit
Enter your choice : 1
sorted array (Bubble sort): 3 12 21 45 78 89

-- Sorting Menu --
1. Bubble sort
2. selection sort
3. Heap Sort
4. Exit
Enter your choice: 2
sorted array (Selection Sort): 3 12 21 45 78 89

-- Sorting Menu --
1. Bubble sort
2. Selection sort
3. Heap Sort
4. Exit
Enter your choice: 3
sorted array (Heap Sort): 3 12 21 45 78 89

```
void heapsort(int a[], int n){
    for(int i = n/2 - 1; i >= 0; i--)
        heapify(a, n, i);
        for(int i = n-1; i > 0; i--){
            swap(&a[0], &a[i]);
            heapify(a, i, 0);
        }
}
int main(){
    int arr[100], t[100], n, ch;
    printf("Enter number of elements:");
    scanf("%d", &n);
    printf("Enter %d elements :", n);
    for(int i=0; i<n; i++)
        scanf("%d", &arr[i]);
do {
    printf("\n -- Sorting Menu --\n
1. Bubble Sort \n 2. Selection Sort \n
3. Heap Sort \n 4. Exit \n Enter your
    choice:");
    scanf("%d", &ch);
    for(int i=0; i<n; i++)
        t[i] = arr[i];
    if(ch==1) { bubblesort(t, n);
    printf("Sorted array (Bubble sort):");
    printArray(t, n); }
        else if(ch==2) { selectionsort(t,n
    printf("Sorted array (selection sort):");
    printArray(t, n); }
        else if(ch==3) { heapsort(t, n);
    printf("Sorted array (Heap Sort):");
    printArray(t, n); }
```

-- Sorting Menu --
1. Bubble sort
2. Selection sort
3. Heap sort
4. Exit
Enter your choice: 4
Exiting program.

```
    else if (ch==4) printf ("Exiting program");
    else printf ("Invalid choice.\n");
    } while (ch!=4);
    return 0;
}
```

* **Result:**

→ Thus, a C program was written and executed successfully to sort an array using Bubble Sort, Selection Sort and Heap Sort.

21/10/26

## Exp: 12: Implementation of Hashing with collision Resolution using linear Probing

* **Aim:** To write a C program to implement a hash table using the division hash function and to resolve collisions using the linear probing technique, and to perform insertion, search and display operations.

* **Algorithm:**

1. Start the program and initialize a hash table array ht[10] with -1 [indicating empty slots].
2. Hash function: compute the home index using h = key % 10.
3. insert (key): probe indices idx = (h + i) for i = 0 to 9.
4. If ht[idx] == key, report duplicate and return.
5. If ht[idx] == -1, store key at ht[idx], report collision if i > 0, and print the inserted index.
6. Search (key): Probe idx = (h+1) % 10. if ht[idx] == key, display the and return. Stop if an empty slo is encountered or all 10 slots are c and report "not found",
7. Display: Print all indices 0 to 9 along their stored keys (or -- if emty). and a with options, repeat until Exit is sele
8. Stop.

* Program:

```
#include<stdio.h>
#define SIZE 10
int ht[SIZE];
void insert(int key) {
    int h = key % SIZE;
    for(int i=0; i<SIZE; i++) {
        int idx = (h+i) % SIZE;
        if(ht[idx] == key) {
            printf("Key %d already exists.",
                                    key);
            return;
        }
        if (ht[idx] == -1) {
            ht[idx] = key;
            if (i > 0) printf("collision occured
            for Key %d.", key);
            printf("Key %d inserted at index %d",
                key, idx);
            return;
        }
    }
    printf("Hash Table is full. cannot
    insert %d", key);
}
void search(int key) {
    int h = key % SIZE;
    for (int i = 0; i < SIZE; i++) {
        int idx = (h+i) % SIZE;
        if (ht[idx] == -1) break;
        if (ht[idx] == key) {
printf("Key %d found at index %d", key, id
```

* Output:

```
-- Hashing (linear Probing) Menu --
1. Insert
2. Search
3. Display
4. Exit
Enter your choice: 1
Enter key to insert: 25
key 25 inserted at index 5

-- Hashing (linear Probing) Menu --
1. Insert
2. Search
3. Display
4. Exit
Enter your choice : 1
Enter key to insert: 35
collision occurred for key 35. key 35
inserted at index 6

-- Hashing (linear Probing) Menu --
1. Insert
2. Search
3. Display
4. Exit
Enter your choice: 1
Enter key to insert: 15
collision occurred for key 15. key 15
inserted at index 7
```

```
    return ;
}}
printf("Key %d not found in hash table.
\n", key);
}
void display() {
  printf("\n Hash Table:\n");
 for (int i=0; i<SIZE; i++){
  if (ht[i] != -1) printf("Index %d:
%d \n", i, ht[i]);
     else printf("Index %d: --\n", i);
   }
}
int main() {
  int ch, key;
  for (int i=0; i<SIZE; i++) ht[i]=-1;
 do {
    printf("\n -- Hashing (linear Probing)
Menu --\n 1. Insert \n 2. Search \n 3.
Display. \n 4. Exit \n Enter your
choice: ");
    scanf("%d", &ch);
  if (ch==1) { printf("Enter key
to insert: "); scanf("%d", &key);
    insert(key); }
  else if (ch==2) { printf("Enter
key to search: "); scanf("%d", &key);
    search(key); }
else if (ch==3) display();
else if (ch == 4) printf("Exiting
program. \n");
```

-- Hashing (linear Probing) Menu --
1. Insert
2. Search
3. Display
4. Exit
Enter your choice : 3

Hash Table:
Index 0 : --
Index 1 : --
Index 2 : --
Index 3 : --
Index 4 : --
Index 5 : 25
Index 6 : 35
Index 7 : 15
Index 8 : --
Index 9 : --

-- Hashing (linear Probing) Menu --
1. Insert
2. Search
3. Display
4. Exit
Enter your choice: 2
Enter Key to Search: 35
Key 35 found at index 6

-- Hashing (linear Probing) Menu --
1. Insert 2. Search 3. Display 4. Exit
Enter your choice : 4
Exiting program.

```
        else printf("Invalid choice.\n");
    } while (ch1 != 4);
        return 0;
}
```

* → **Result:**

Thus, a C program was written and executed successfully to implement hashing with collision resolution using linear probing.